```
  return (
    <td>
      <ButtonSquare status={status} onClick={changeState}>
        {seatNumber + 1}
      </ButtonSquare>
    </td>
  )
}

export default Seat
```

There are a few different things going on here. Let's start by looking at the definition of ButtonSquare in the middle of the file.

This use of styled-components adds a couple of new features. First, we start with a call to styled.span, but we chain a new method there, resulting in styled.span.attrs({ className: buttonClass }), where buttonClass is a previously defined list of our Tailwind utility classes. The attrs method takes as an argument a JavaScript object and merges that argument into the props as they are passed to the component. In this case, we're passing className as a prop, which React will interpret as a CSS class, so we get the existing styling. If you want to be a little fancier, the attrs method could also take as its argument a function (that function's argument is the props passed to the component and the return value is the attributes to be merged into the props).

Having ensured button styling, we now want to make sure the background color changes. In the previous version, the Seat had a function that converted status into color. In this version, the Seat is passing the status to the ButtonSquare styled component and the styled component is responsible for converting that status into a color.

First, the ButtonSquare is a component getting a prop, so we want to have a TypeScript interface to ensure that the prop has the expected attributes. We define SquareProps to take the status being passed in and the classNames being added in our attrs method. (The styled-attributes library says that if you use className in a props interface, it needs to be optional because of how the library works, so